

Deep Dive — Module 7: Tesseract OCR (Text from Images)

***Goal:** understand the fallback that makes scanned and photographed documents searchable — what OCR is, how it works, how the project calls Tesseract from Java (Tess4J), how the digit whitelist boosts financial accuracy, and how low-confidence results are handled.*

Where it sits: OCR is the **safety net on the input side**. Digital PDFs and DOCX files give clean text directly (Module 9). But many financial documents are *scanned paper* — for those, text extraction returns empty, and Tesseract reads the pixels to recover the text so it can be chunked, embedded, and indexed like any other document.

1. What OCR is and why the project needs it

OCR (Optical Character Recognition) converts an image of text into machine-readable text. The pipeline can't embed what it can't read:

Digital PDF → PDFBox extracts text  → use it directly
Scanned PDF → PDFBox returns ""  → trigger Tesseract OCR
Photo (JPG/PNG/TIFF) → can't parse  → trigger Tesseract OCR

OCR is therefore a **conditional fallback**: it runs *only when* normal extraction yields empty text. This keeps the fast path fast and reserves the slower OCR (5–8 s per page) for documents that actually need it.

2. How OCR works (conceptually)

Image

- 1. Preprocessing: binarize (B/W), skew (straighten), denoise
 - 2. Layout analysis: find text blocks, lines, words
 - 3. Character recognition: match each glyph shape to a known character
 - 4. Post-processing: assemble characters → words → lines
- "INVOICE Acme Corp INV-2048 Total: \$1,250.75"

Preprocessing matters most: clean, high-contrast, upright images recognize far better than skewed, noisy scans. The project binarizes and deskews via `BufferedImage` before handing the image to Tesseract.

3. Calling Tesseract from Java (Tess4J)

Tess4J is a Java wrapper (JNI bindings) around the native Tesseract engine — so OCR runs **in-process** inside the Spring Boot backend, no separate service.

```
import net.sourceforge.tess4j.ITesseract;
import net.sourceforge.tess4j.Tesseract;

public class OCRService {
    public String extractText(MultipartFile file) throws Exception {
        BufferedImage image = ImageIO.read(file.getInputStream()); // file → image
        ITesseract tess = new Tesseract();
        tess.setDatapath("/usr/share/tesseract-ocr/4.00/tessdata"); // language data path
        tess.setLanguage("eng"); // English
        return tess.doOCR(image); // run OCR
    }
}
```

- `setDatapath` points to the `tessdata` folder containing trained language models.
 - `setLanguage("eng")` selects English; other packs (39 languages) can be added (Phase 2 in the PRD).
 - Because Tesseract is a *native* dependency, it must be installed on the host or bundled in the backend's Docker image / as a sidecar.
-

4. Digit whitelist (financial accuracy)

OCR commonly confuses look-alike characters: 0 ↔ Ø, 1 ↔ l, S ↔ 5. For financial fields (amounts, invoice numbers), this is dangerous. A **whitelist** restricts recognition to expected characters:

```
tesseract.setVariable("tessedit_char_whitelist", "0123456789.,$/€£-INV");  
// Without: "lNV-2048 $1,250.75" (confused letters)  
// With:    "INV-2048 $1,250.75" (correct)
```

Use whitelists selectively — applying a digits-only whitelist to a paragraph of vendor description would mangle it. The project applies tighter character sets to *financial fields* and broader recognition to free text.

5. Confidence & fallback handling

OCR can fail on poor scans. The service checks whether the result is plausible and degrades gracefully:

```
String ocrText = tesseract.doOCR(image);  
if (ocrText == null || ocrText.trim().length() < 10) {  
    document.setProcessingStatus("failed");  
    alertService.createAlert("OCR_LOW_CONFIDENCE", document.getId());  
}
```

The Document Detail screen can show a **per-field confidence badge** (HIGH/MEDIUM/LOW) so a human knows which extracted values to double-check. This connects to the PRD's open question about a minimum OCR-accuracy go/no-go threshold before relying on it in production. Operationally, an OCR failure rate above ~20% in a 5-minute window raises a monitoring alert.

6. Where OCR fits the upload pipeline

```
extractText(file):  
  .pdf → PDFBox; if empty → ocrService.extractText(file)  
  .docx → POI  
  .txt → read bytes  
  image → ocrService.extractText(file)  // always OCR
```

After OCR produces text, it rejoins the normal flow: clean → chunk (~800 chars) → embed → upsert to Qdrant → persist metadata (`ocr_used = true`) and audit. The `ocr_used` flag is stored so you can later audit which documents relied on OCR.

7. Common pitfalls

- **Running OCR on every file** — it's slow; only use it when normal extraction is empty.
 - **Skipping preprocessing** — skewed/noisy images wreck accuracy; binarize and deskew first.
 - **Whitelisting too aggressively** — digits-only on prose destroys vendor names; scope whitelists to fields.
 - **Forgetting the native dependency** — Tess4J needs the Tesseract engine + `tessdata` installed/bundled.
 - **Trusting OCR output blindly** — check confidence; flag low-confidence docs for human review.
 - **Not recording `ocr_used`** — you lose the audit signal of which data came from imperfect OCR.
 - **Assuming one language** — multilingual scans need the right language packs.
-

8. Practice exercises

Install Tesseract (`apt-get install tesseract-ocr`) and Tess4J (Maven), or experiment with the `tesseract` CLI first.

1. Run the CLI: `tesseract sample_invoice.png out` and inspect `out.txt`.

2. Take a clean screenshot of an invoice and a deliberately skewed/blurry one; compare OCR quality.
 3. Binarize + deskew the bad image (any image tool) and re-run; note the improvement.
 4. In Java/Tess4J, OCR an image and print the text.
 5. Apply the financial whitelist and OCR an image containing `INV-2048 $1,250.75` ; compare with/without.
 6. Implement the low-confidence check (`length < 10` → mark failed) and trigger it on a near-blank image.
 7. Write the routing logic so `.pdf` uses PDFBox first and only falls back to OCR when empty.
 8. (Stretch) Add a second language pack and OCR a non-English sample.
-

9. Self-check questions

- When exactly does OCR run in this pipeline, and why not always?
 - What are the four conceptual stages of OCR, and which most affects accuracy?
 - Why does Tess4J make OCR an in-process step rather than a separate service?
 - How does the digit whitelist improve financial-field accuracy, and when is it harmful?
 - How does the system handle a scan it can't read well?
 - Why store the `ocr_used` flag?
 - What native dependency must be present for Tess4J to work?
-

10. Glossary

- **OCR** — Optical Character Recognition; image → text.
- **Tesseract** — open-source OCR engine.
- **Tess4J** — Java (JNI) wrapper for Tesseract.
- **tessdata** — folder of trained language models.
- **Binarization** — converting to black/white for cleaner recognition.
- **Deskew** — straightening tilted text.
- **Whitelist** (`tessedit_char_whitelist`) — restrict recognized characters.

- **Confidence** — model's certainty about recognized text.

Navigation: [← Module 6 Docker](#) | [Index](#) | [Module 8 JWT →](#)